

Uni-App 分类列表组件（含页面跳转功能）

分类列表是App与小程序的核心导航组件，用于将内容或功能按类别划分，方便用户快速定位目标页面。本文将实现一个“数据驱动、样式统一、多端适配”的分类列表组件，支持网格布局与列表布局切换，点击分类项可直接跳转至对应页面，同时提供未登录状态拦截、跳转动画等增强功能。

一、核心设计思路

1. 功能定位

分类列表需满足“导航精准、交互流畅、扩展灵活”三大核心需求，具体功能包括：

- **分类展示**：支持图标+文字组合展示，清晰区分各类别；
- **页面跳转**：点击分类项通过`navigator`组件或`uni.navigateTo`实现页面跳转，支持配置跳转类型；
- **状态拦截**：针对需要登录的分类项，未登录时弹窗提示并跳转至登录页；
- **布局切换**：支持网格（默认3列）与列表（1列）两种布局，适应不同场景；
- **多端适配**：在微信小程序、App、H5端保持样式与交互一致性。

2. 数据驱动设计

采用“数据配置+组件渲染”的模式，分类列表的内容、图标、跳转地址均通过数组配置，新增或修改分类时无需改动组件代码，仅需更新配置数据。数据结构如下：

Code block

```
1  // 分类列表配置数据示例
2  categoryList: [
3    {
4      id: 1,                // 唯一标识
5      name: "我的订单",     // 分类名称
6      icon: "/static/icons/order.png", // 分类图标
7      url: "/pages/order/order", // 跳转页面路径
8      needLogin: true,     // 是否需要登录
9      badge: 3,            // 未读消息数（可选）
10     type: "navigateTo"    // 跳转类型 (navigateTo/redirectTo/switchTab)
11   },
12   // 更多分类项...
13 ]
```

3. 技术选型

- **布局方案**：使用Flex布局实现网格与列表切换，通过动态绑定class控制布局样式；
- **跳转方式**：优先使用Uni-App原生`navigator`组件，复杂场景（如登录拦截）使用`uni.navigateTo`等API；
- **状态管理**：登录状态通过全局变量或缓存获取，无需引入复杂状态管理库；
- **样式方案**：使用SCSS编写样式，通过变量统一控制颜色与尺寸，提升可维护性。

二、完整实现代码

1. 分类列表组件（components/CategoryList/CategoryList.vue）

组件支持传入分类数据、布局类型等属性，实现分类展示与跳转核心功能，包含登录拦截逻辑。

Code block

```
1  <template>
2    <view class="category-container">
3
4      <view class="category-title" v-if="title">
5        <text class="title-text">{{title}}</text>
6        <view class="layout-switch" @click="switchLayout" v-
7          if="supportLayoutSwitch">
8          <uni-icons type="grid" size="24rpx" :color="layoutType === 'grid' ?
9            activeColor : defaultColor"></uni-icons>
10         <uni-icons type="list" size="24rpx" :color="layoutType === 'list' ?
11           activeColor : defaultColor"></uni-icons>
12       </view>
13     </view>
14
15     <view class="category-list" :class="{ 'grid-layout': layoutType === 'grid',
16       'list-layout': layoutType === 'list' }">
17       <navigator
18         class="category-item"
19         v-for="item in categoryList"
20         :key="item.id"
21         :url="item.needLogin ? (isLogin ? item.url : '') : item.url"
22         :open-type="item.type || 'navigateTo'"
23         @click="handleItemClick(item)"
24       >
25
26       <view class="item-icon">
27         <image :src="item.icon" mode="widthFix"></image>
```

```
26         <view class="badge" v-if="item.badge">
27             <text>{{item.badge > 9 ? '9+' : item.badge}}</text>
28         </view>
29     </view>
30
31
32     <view class="item-text">{{item.name}}</view>
33
34
35     <uni-icons class="item-arrow" type="right" size="22rpx" color="#999" v-
if="layoutType === 'list'"></uni-icons>
36 </navigator>
37 </view>
38 </view>
39 </template>
40
41 <script>
42 import uniIcons from '@dcloudio/uni-ui/lib/uni-icons/uni-icons.vue';
43
44 export default {
45     components: { uniIcons },
46     // 组件属性（支持外部配置）
47     props: {
48         title: {
49             type: String,
50             default: '' // 组件标题，为空时不显示
51         },
52         categoryList: {
53             type: Array,
54             required: true, // 分类列表数据（必传）
55             validator: (value) => {
56                 // 验证数据结构，确保每个分类项包含必要字段
57                 return value.every(item => item.id && item.name && item.url);
58             }
59         },
60         supportLayoutSwitch: {
61             type: Boolean,
62             default: true // 是否支持布局切换
63         },
64         defaultLayout: {
65             type: String,
66             default: 'grid', // 默认布局: grid (网格) /list (列表)
67             validator: (value) => {
68                 return ['grid', 'list'].includes(value);
69             }
70         },
71         activeColor: {
```

```

72     type: String,
73     default: '#FF5722' // 激活状态颜色（如选中的布局图标）
74   },
75   defaultColor: {
76     type: String,
77     default: '#999999' // 默认颜色
78   }
79 },
80 data() {
81   return {
82     layoutType: this.defaultLayout, // 当前布局类型
83     isLogin: false // 登录状态（实际开发中从缓存或全局获取）

```

2. 父组件使用示例（pages/index/index.vue）

在首页中引入分类列表组件，配置分类数据与跳转路径，实现页面导航功能。

Code block

```

1  <template>
2    <view class="index-page">
3
4      <CategoryList
5        title="服务分类"
6        :categoryList="categoryList"
7        defaultLayout="grid"
8        @layoutChange="onLayoutChange"
9      ></CategoryList>
10   </view>
11 </template>
12
13 <script>
14 // 引入分类列表组件
15 import CategoryList from '@components/CategoryList/CategoryList.vue';
16
17 export default {
18   components: { CategoryList },
19   data() {
20     return {
21       // 分类列表配置：每个项对应一个跳转页面
22       categoryList: [
23         {
24           id: 1,
25           name: "我的订单",
26           icon: "/static/icons/order.png",
27           url: "/pages/order/order",
28           needLogin: true,
29           badge: 3,

```

```
30         type: "navigateTo"
31     },
32     {
33         id: 2,
34         name: "地址管理",
35         icon: "/static/icons/address.png",
36         url: "/pages/address/address",
37         needLogin: true,
38         type: "navigateTo"
39     },
40     {
41         id: 3,
42         name: "会员中心",
43         icon: "/static/icons/member.png",
44         url: "/pages/member/member",
45         needLogin: true,
46         type: "navigateTo"
47     },
48     {
49         id: 4,
50         name: "优惠券",
51         icon: "/static/icons/coupon.png",
52         url: "/pages/coupon/coupon",
53         needLogin: true,
54         badge: 5,
55         type: "navigateTo"
56     },
57     {
58         id: 5,
59         name: "客服中心",
60         icon: "/static/icons/customer.png",
61         url: "/pages/customer/customer",
62         needLogin: false, // 无需登录即可访问
63         type: "navigateTo"
64     },
65     {
66         id: 6,
67         name: "关于我们",
68         icon: "/static/icons/about.png",
69         url: "/pages/about/about",
70         needLogin: false,
71         type: "navigateTo"
72     }
73 ]
74 };
75 },
76 methods: {
```

```
77 // 监听分类列表布局切换事件（可选）
78 onLayoutChange(layoutType) {
79   console.log("当前布局类型：", layoutType);
80   // 可根据布局类型执行额外逻辑（如统计用户偏好）
81 }
82 }
83 }
84 </script>
85
86 <style scoped lang="scss">
87 .index-page {
88   background-color: $neutral-bg;
```

3. 跳转目标页面示例（pages/order/order.vue）

分类项跳转的目标页面，此处以“我的订单”页面为例，展示跳转后的页面效果。

Code block

```
1 <template>
2   <view class="order-page">
3     <view class="page-header">
4       <text class="header-title">我的订单</text>
5     </view>
6     <view class="order-content">
7       <text class="empty-tip" v-if="orderList.length === 0">暂无订单，快去下单吧~
      </text>
8
9     </view>
10  </view>
11 </template>
12
13 <script>
14 export default {
15   data() {
16     return {
17       orderList: [] // 订单数据，实际开发中从接口获取
18     };
19   },
20   onLoad() {
21     // 页面加载时获取订单数据
22     this.getOrderList();
23   },
24   methods: {
25     getOrderList() {
26       // 模拟接口请求
27       uni.request({
28         url: "https://your-server/api/order/list",
```

```

29      data: {
30        userId: uni.getStorageSync('userInfo').userId
31      },
32      success: (res) => {
33        this.orderList = res.data.data;
34      }
35    });
36  }
37 }
38 }
39 </script>
40
41 <style scoped lang="scss">
42 .order-page {
43   background-color: $neutral-bg;
44   min-height: 100vh;
45   padding: 20rpx;
46   box-sizing: border-box;
47 }
48 .page-header {
49   margin-bottom: 30rpx;
50 }
51 .header-title {
52   font-size: 32rpx;
53   font-weight: 600;
54   color: $neutral-black;
55 }
56 .empty-tip {
57   display: block;
58   text-align: center;
59   margin-top: 100rpx;
60   color: $neutral-light-gray;
61   font-size: 24rpx;
62 }
63 </style>

```

4. 登录页与返回逻辑（pages/login/login.vue）

实现登录功能，并在登录成功后返回之前拦截的分类目标页面。

Code block

```

1  <template>
2    <view class="login-page">
3      <view class="login-form">
4        <view class="form-item">
5          <text class="label">手机号</text>

```

```
6      <input v-model="phone" type="number" placeholder="请输入手机号">
7    </view>
8    <view class="form-item">
9      <text class="label">验证码</text>
10     <input v-model="code" type="number" placeholder="请输入验证码">
11     <button class="get-code-btn" @click="getCode">获取验证码</button>
12   </view>
13   <button class="login-btn" @click="handleLogin">登录</button>
14 </view>
15 </view>
16 </template>
17
18 <script>
19 export default {
20   data() {
21     return {
22       phone: "",
23       code: "",
24       redirectUrl: "" // 登录后需要跳转的目标页面
25     };
26   },
27   onLoad(options) {
28     // 获取分类列表组件传递的跳转地址
29     if (options.redirectUrl) {
30       this.redirectUrl = decodeURIComponent(options.redirectUrl);
31     }
32   },
33   methods: {
34     // 获取验证码（模拟）
35     getCode() {
36       if (!/^1[3-9]\d{9}$/.test(this.phone)) {
37         uni.showToast({ title: "请输入正确的手机号", icon: "none" });
38         return;
39       }
40       uni.showToast({ title: "验证码已发送", icon: "none" });
41     },
42
43     // 登录处理
44     handleLogin() {
45       if (!this.phone || !this.code) {
46         uni.showToast({ title: "请完善登录信息", icon: "none" });
47         return;
48       }
49
50       // 模拟登录接口请求
51       uni.request({
52         url: "https://your-server/api/login",
```



```
53         method: "POST",
54         data: { phone: this.phone, code: this.code },
55         success: (res) => {
56             if (res.data.code === 0) {
57                 // 登录成功: 保存用户信息到缓存
58                 uni.setStorageSync('userInfo', res.data.data);
59
60                 // 跳转到之前拦截的目标页面
61                 if (this.redirectUrl) {
62                     uni.navigateTo({ url: this.redirectUrl });
63                 } else {
64                     // 无目标页面则返回上一页
65                     uni.navigateBack();
66                 }
67             } else {
68                 uni.showToast({ title: res.data.msg, icon: "none" });
69             }
70         }
71     });
72 }
73 }
74 }
75 </script>
```

三、核心功能详解

1. 页面跳转机制

分类列表支持Uni-App所有跳转类型，通过`type`属性配置，核心跳转方式包括：

跳转类型	适用场景	配置示例
navigateTo	普通页面跳转，保留当前页面（有返回按钮）	type: "navigateTo"
redirectTo	关闭当前页面，跳转到目标页面（无返回按钮）	type: "redirectTo"
switchTab	跳转到TabBar页面，关闭其他非TabBar页面	type: "switchTab"
reLaunch	关闭所有页面，跳转到目标页面	type: "reLaunch"

默认使用`navigateTo`，适合大多数分类跳转场景；若分类项对应TabBar页面（如“首页”“我的”），需将`type`设为`switchTab`。

2. 登录状态拦截

针对需要登录的分类项（`needLogin: true`），实现完整的拦截与跳转逻辑：

1. 点击分类项时，组件先判断`isLogin`状态（从缓存获取）；
2. 已登录：通过`navigator`组件直接跳转到目标页面；
3. 未登录：拦截`navigator`跳转（将`url`设为空），弹出模态框提示用户登录；
4. 用户确认登录后，跳转至登录页，并携带目标页面路径（`redirectUrl`）；
5. 登录成功后，自动跳转到之前点击的目标页面，实现“登录-返回”无缝衔接。

3. 布局切换功能

支持网格与列表两种布局，通过动态绑定class实现样式切换：

- **网格布局**：默认3列，适合展示多个分类项，节省页面空间；
- **列表布局**：1列，每个分类项包含图标、文字与箭头，适合分类项较多的场景；
- **切换逻辑**：点击标题栏的布局图标触发切换，通过`\$emit`事件向父组件传递当前布局类型，支持父组件监听。

4. 未读消息徽章

分类项支持配置`badge`属性，实现未读消息提示功能：

- `badge`值为数字时，显示对应未读数量；
- 数量超过9时，自动显示为“9+”，避免徽章过大影响布局；
- 徽章采用绝对定位附着在图标右上角，符合用户视觉习惯。

四、多端适配与优化技巧

1. 多端适配要点

- **尺寸适配**：所有尺寸使用rpx单位，自动适配不同屏幕宽度；
- **H5端**：限制组件最大宽度为600rpx，避免在PC端页面过宽导致布局松散；
- **App端**：通过`::v-deep`穿透样式隔离（如需修改子组件样式），添加硬件加速提升渲染流畅度；
- **微信小程序**：优化`navigator`组件跳转性能，避免在复杂列表中频繁使用`wx.navigateTo` API。

2. 性能优化建议

- **图片懒加载**：分类图标添加`lazy-load="true"`属性，减少页面初始加载压力；

- **数据缓存**：分类列表数据若从接口获取，建议缓存到本地，避免每次页面加载都请求接口；
- **避免频繁重绘**：布局切换时仅修改class，不直接操作DOM，减少页面重绘次数；
- **组件复用**：分类列表组件封装为通用组件，在个人中心、首页等多个页面复用，提升开发效率。

3. 交互体验优化

- **点击反馈**：`navigator` 组件自带轻微的点击反馈（灰色背景），无需额外添加；
- **加载状态**：跳转页面时，Uni-App自动显示加载动画，无需手动添加；
- **错误处理**：若跳转页面路径错误，通过`uni.getSystemInfo`结合`try-catch`捕获错误，提示用户“页面不存在”；
- **无障碍适配**：为分类项添加`aria-label`属性，提升无障碍访问体验。

五、扩展功能与场景

1. 动态分类数据（从接口获取）

实际开发中分类列表常从后端接口获取，可在父组件中修改数据获取逻辑：

Code block

```
1  onLoad() {
2    // 从接口获取分类数据
3    this.getCategoryList();
4  },
5  methods: {
6    getCategoryList() {
7      uni.showLoading({ title: "加载中" });
8      uni.request({
9        url: "https://your-server/api/category/list",
10       success: (res) => {
11         if (res.data.code === 0) {
12           this.categoryList = res.data.data;
13         } else {
14           uni.showToast({ title: res.data.msg, icon: "none" });
15         }
16       },
17       fail: () => {
18         uni.showToast({ title: "网络错误，加载失败", icon: "none" });
19       },
20       complete: () => {
21         uni.hideLoading();
22       }
23     });
24   }
25 }
```

2. 分类项禁用状态

如需添加禁用的分类项（如功能维护中），可扩展数据结构与组件逻辑：

Code block

```
1  // 扩展分类项数据
2  {
3    id: 7,
4    name: "积分商城",
5    icon: "/static/icons/points.png",
6    url: "/pages/points-mall/mall",
7    needLogin: true,
8    disabled: true, // 禁用状态
9    disabledTips: "功能维护中，敬请期待"
10 }
11
12 // 组件中添加禁用样式与逻辑
13 // template部分
14 <view class="category-item" :class="{disabled: item.disabled}">
15
16 </view>
17
18 // script部分
19 handleItemClick(item) {
20   if (item.disabled) {
21     uni.showToast({ title: item.disabledTips, icon: "none" });
22     return;
23   }
24   // 原有登录拦截逻辑...
25 }
26
27 // style部分
28 .disabled {
29   opacity: 0.5;
30   pointer-events: none; // 禁止点击
31 }
```

3. 自定义分类项样式

通过组件属性传递自定义样式，实现不同页面的分类列表差异化展示：

Code block

```
1  <!-- 父组件传递自定义样式 -->
```

```
2 <CategoryList
3   :itemStyle="{backgroundColor: '#F5F5F5', borderRadius: '16rpx'}"
4 ></CategoryList>
5
6 <!-- 组件中绑定自定义样式 -->
7 <navigator class="category-item" :style="itemStyle">
8
9 </navigator>
```

六、总结：分类列表开发核心要点

分类列表作为页面导航的核心组件，开发时需兼顾“功能性、可维护性、用户体验”三大维度，核心要点总结如下：

1. **数据驱动**：通过配置数组管理分类项，实现“数据修改即更新UI”，提升开发与维护效率；
2. **跳转精准**：根据页面类型选择合适的跳转方式，配合登录拦截确保功能权限可控；
3. **多端兼容**：使用rpx单位与条件编译，确保在不同平台展示与交互一致；
4. **体验优化**：添加布局切换、未读提示、点击反馈等细节，提升用户使用体验；
5. **扩展灵活**：预留禁用状态、自定义样式等扩展点，适应不同业务场景。



实际开发中，可根据App的业务属性调整分类项内容与样式：电商类App可添加“商品分类”“购物车”等项；工具类App可添加“常用功能”“设置”等项，确保分类列表与业务高度匹配。

（注：文档部分内容可能由 AI 生成）